Practical – Week8

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
  GNU nano 8.7.1                                dynamic.c *
#include <stdio.h>
#include <stdlib.h>
int main() {
    int i;
    int *a, *b, *temp;
    // malloc()
    a = (int *)malloc(3 * sizeof(int));
    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    for (i = 0; i < 3; i++) {
        a[i] = i + 1;
    }
    printf("malloc values: ");
    for (i = 0; i < 3; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    // calloc()
    b = (int *)calloc(3, sizeof(int));
    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }
    printf("calloc values: ");
    for (i = 0; i < 3; i++) {
        printf("%d ", b[i]);
    }
    printf("\n");
    // realloc()
    temp = (int *)realloc(a, 5 * sizeof(int));
    if (temp == NULL) {
        printf("realloc failed\n");
        free(a);
        free(b);
```

```
manikanta@Hemu:~/OS_Lab$ nano dynamic.c
manikanta@Hemu:~/OS_Lab$ gcc -Wall -g dynamic.c -o dynamic
manikanta@Hemu:~/OS_Lab$ ./dynamic
malloc values: 1 2 3
calloc values: 0 0 0
After realloc: 1 2 3 4 5
Memory freed successfully.
```

```
manikanta@Hemu:~/OS_Lab$ valgrind --leak-check=full ./dynamic
==1017== Memcheck, a memory error detector
==1017== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1017== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright inf
==1017== Command: ./dynamic
==1017==
malloc values: 1 2 3
calloc values: 0 0 0
After realloc: 1 2 3 4 5
Memory freed successfully.
==1017==
==1017== HEAP SUMMARY:
==1017==     in use at exit: 0 bytes in 0 blocks
==1017==   total heap usage: 4 allocs, 4 frees, 1,068 bytes allocated
==1017==
==1017== All heap blocks were freed -- no leaks are possible
==1017==
==1017== For lists of detected and suppressed errors, rerun with: -s
==1017== ERROR SUMMRY: 0 errors from 0 contexts (suppressed: 0 from 0)
manikanta@Hemu:~/OS_Lab$
```

```
  GNU nano 8.7.1                                leak.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;

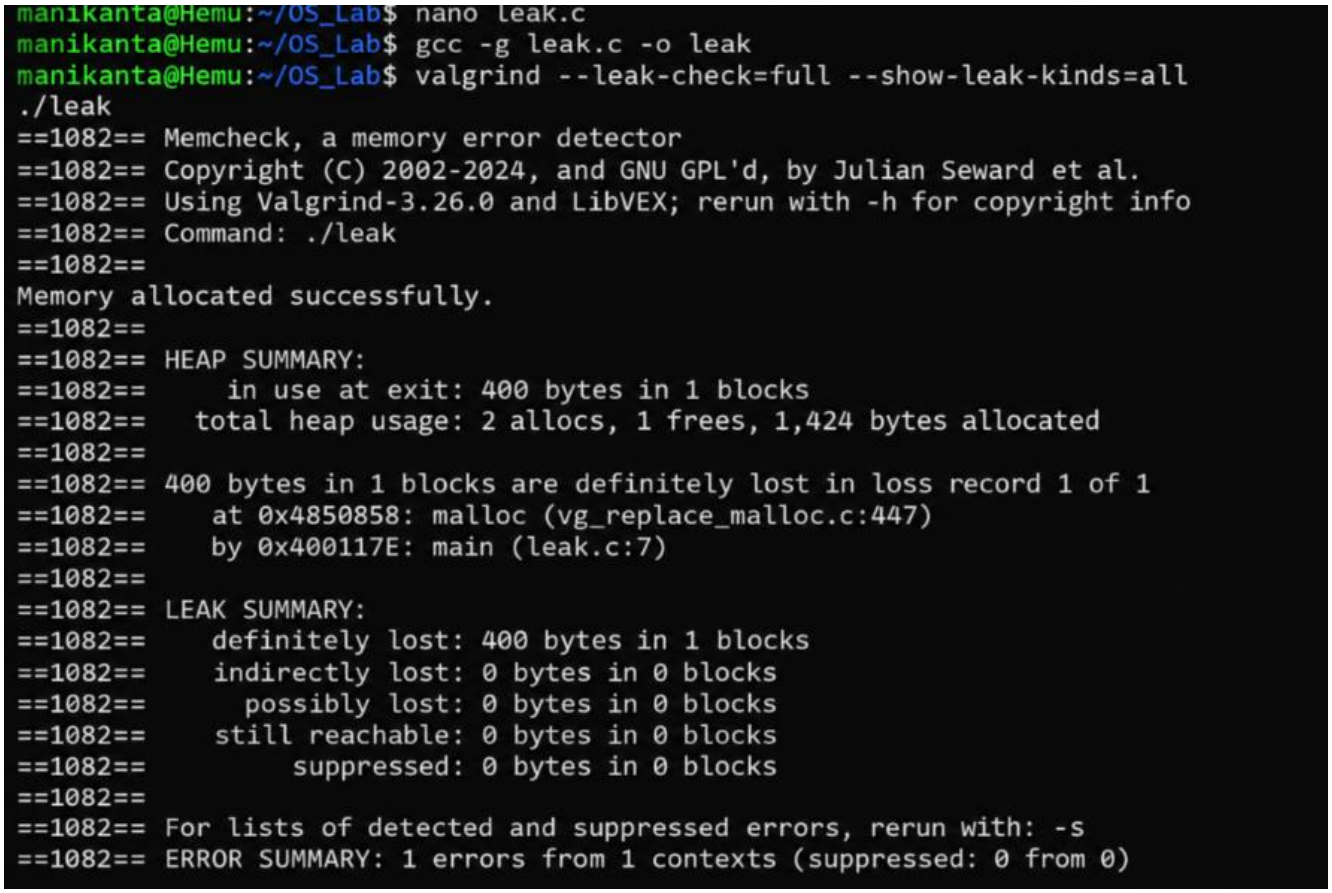
    ptr = (int *)malloc(100 * sizeof(int));

    if (ptr == NULL) {
        return 1;
    }

    printf("Memory allocated successfully.\n");

    // Intentionally not freeing ptr

    return 0;
}
```



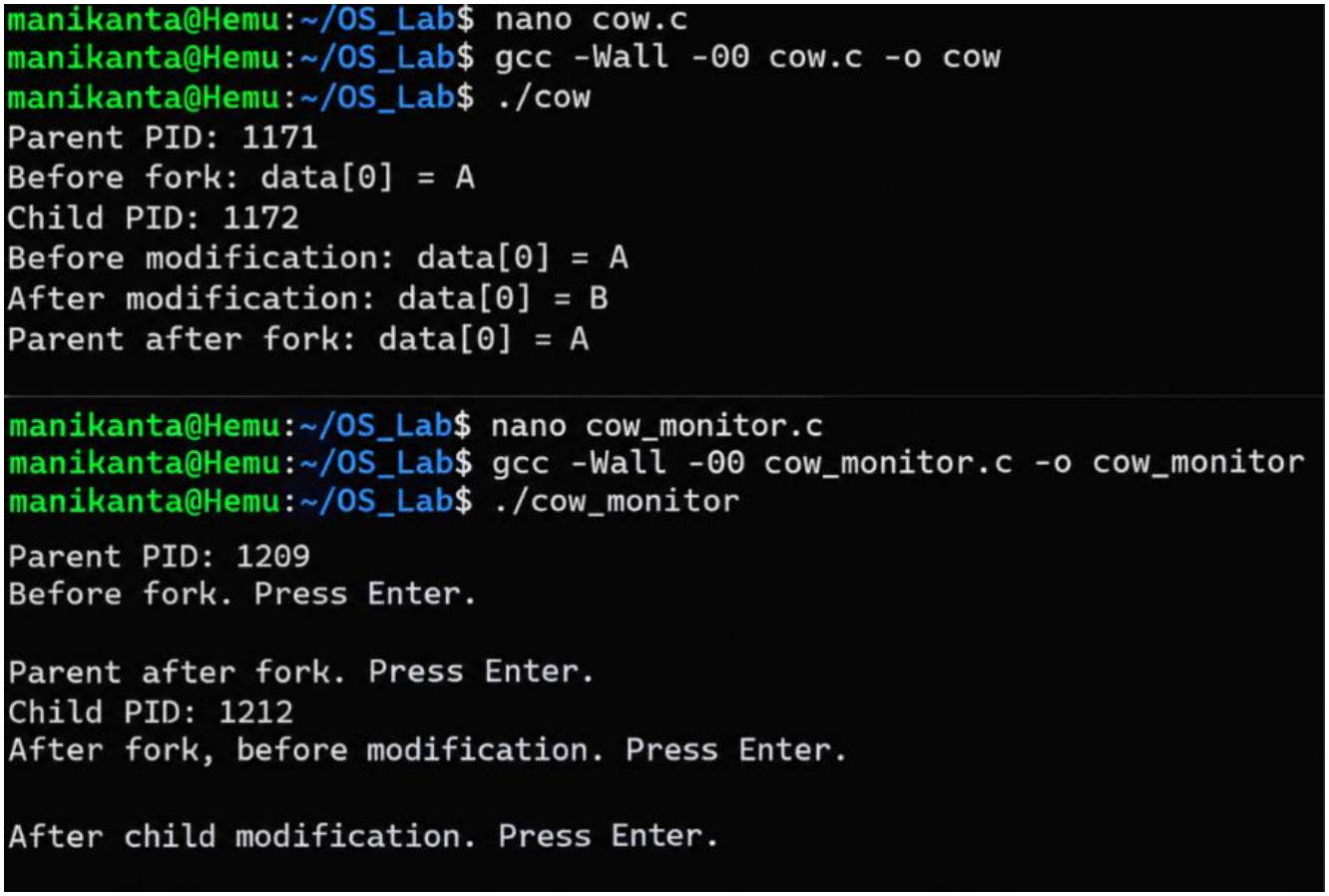
Report: Dynamic memory allocation allows programs to use memory during execution as needed. Valgrind helps identify memory leaks and invalid memory usage, improving program reliability and memory management.

2. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
  GNU nano 8.7.1                                cow.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

#define SIZE (10 * 1024 * 1024)

int main() {
    char *data;
    pid_t pid;
    data = malloc(SIZE);
    if (data == NULL) {
        perror("malloc");
        return 1;
   }
    // Touch all pages so physical memory is allocated
    for (int i = 0; i < SIZE; i += 4096) {
        data[i] = 'A';
    }
    printf("Parent PID: %d\n", getpid());
    printf("Before fork: data[0] = %c\n", data[0]);
    pid = fork();
    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }
    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        printf("Before modification: data[0] = %c\n", data[0]);
        // Modify shared memory
        data[0] = 'B';
        printf("After modification: data[0] = %c\n", data[0]);
        sleep(5);
        free(data);
        exit(0);
```



Report: Copy-on-Write improves memory efficiency by allowing parent and child processes to share pages after fork(). Private copies are created only when a process modifies shared data, reducing unnecessary memory allocation.